

A Data-Centric AI Approach to Building Energy Performance Prediction

The project assignment report of the course of Enterprise AI, SS26.

Xiaojie Gu, Geng Tian, Tiancheng Ma, Thê Viêt Nguyễn, Gunther Gust
University of Würzburg, Germany

Abstract

We extend the skeleton pipeline for predicting building Greenhouse Gas (GHG) emissions on the Chicago Energy Benchmarking dataset along two dimensions. Our **primary** improvement is data-centric feature engineering: we iterate from a one-feature baseline through five versions (v0–v4) that add data leakage filtering, discretization, normalization, and feature importance–driven selection. Our **secondary** improvement is systematic experiment tracking with Weights & Biases (WandB): every version is logged as a named run, so design choices are made by reading a dashboard, not console output. The final v4 model reaches a test R^2 of 0.6882 versus the baseline’s 0.5323, and WandB caught a target-leakage bug in v0 ($R^2 \approx 0.97$) that would otherwise have been reported as a “good result”.

1. Baseline Description

The baseline preprocessing keeps only one continuous feature to predict emissions; it works end-to-end but tops out at test $R^2 = 0.5323$ and MAE = 869.86 tonnes CO₂e (Table 1). We therefore improve the baseline in two connected parts: first, a **semi-feature-agnostic feature engineering** pipeline that avoids hard-coding column names in preprocessing and instead applies automated rules for data deletion handling and particularly feature selection based on importance; second, a **WandB-based experiment tracking** workflow that records every iteration as a named run, so each new preprocessing update can be compared against previous versions using the same metrics.

2. Primary Improvement: Feature Engineering

We treat preprocessing as the main work and iterate it in five versions. Each version(except for the first version) adds one technique; The WandB dashboard (Section 3) records the effect of every step. The mind map in Fig. 1 groups the techniques into three levels: **cell-level** (per-cell ops, e.g. imputation), **column-level** (uniform per-column ops, e.g. nor-

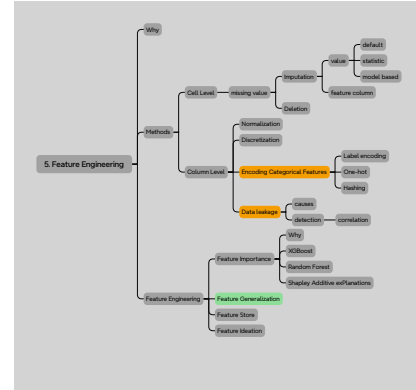


Figure 1. Preprocessing techniques organised by scope: cell, column, dataset. This is the mind map we took notes on from the lecture, which covered most of the technology the lecture told us about feature engineering. We organize the technology into three types: 1. Cell level Working on a single cell (for example, filling the value. Technically, deletion covers some work that should be counted as column level, but another part of it does not – row-wise deletion, so we place it under the cell level). 2. Column level Working on each feature column. For example, normalization involves performing the same operation for each cell in a column. 3. For the dataset level, it refers to feature importance detection.

malization), and **dataset-level** (whole-dataset ops, e.g. feature importance detection).

2.1. v0: All Numeric Features (the trap)

What. We drop the single-feature restriction and feed every numeric and categorical column to OLS, plus some cleaning: drop rows and columns with too many NaNs(Fig. 2), imputation, categorical-numeric transform.

Why. The README hints that “using more features might improve R^2 ”. v0 is the obvious first try.

How. v0 jumps to test $R^2 \approx 0.977$ – a result we initially celebrated, then questioned. The gap to the baseline (+0.445) is implausibly large for “just add columns”, which is exactly what motivates v1. v0 is logged and *kept* in the dashboard so the leakage story is reproducible, but it is rejected as a deliverable.

2.2. v1: Target Leakage Filtering

What. We drop columns that algebraically encode the label. Total GHG is a deterministic linear combination of energy inputs:

$$\text{Total GHG} = \sum_i \text{Energy}_i \times \text{Emission Factor}_i, \quad (1)$$

so any of energy-related features lets the model reconstruct the formula. After reviewing the dataset, we identified the following energy-related columns: `electricity_use_kbtu`, `natural_gas_use_kbtu`, `district_steam_use_kbtu`, `district_chilled_water_use_kbtu`, `all_other_fuel_use_kbtu`, `site_eui_kbtu_sq_ft`, `ghg_intensity_kg_co2e_sq_ft`.

These variables are either direct measures of energy consumption or quantities derived from energy usage or greenhouse gas emissions. Since the prediction target itself is computed from energy consumption, retaining these features would allow the model to access information that is effectively part of the label, resulting in data leakage. Therefore, all of these columns were removed from the dataset.

We also drop those features because they are unavailable at inference time:

`source_eui_kbtu_sq_ft`: the total energy consumed by the whole energy support chain for the site to consume corresponding energy, which can only be obtained after knowing the value of the label.

`water_use_kgal`: this is the value of water usage of the data year, which might not be available during inference.

`chicago_energy_rating`: we need to know the energy consumed by the property before we can calculate the rating.

`energy_star_score`: we need to know the energy consumed by the property before we can calculate the score.

We keep `weather_normalized_source_eui_kbtu_sq_ft`: According to the official definition of these two columns, it means they might use the weather conditions over the past 30 years to predict the energy that will be consumed. Therefore, they might not contain information directly from the label, and it is usually available when predicting a new property's greenhouse gas emissions.

Why. Without this step v0's $R^2 = 0.977$ would ship a model that fails the moment the energy columns are unavailable at inference – which is always, by construction.

If a feature has a deterministic or near-deterministic mathematical relationship with the target variable (e.g., through a known formula or functional dependency), it is often a strong indicator of potential data leakage and should usually be excluded. In general, whenever a feature exhibits an unusually strong relationship with the target, its origin

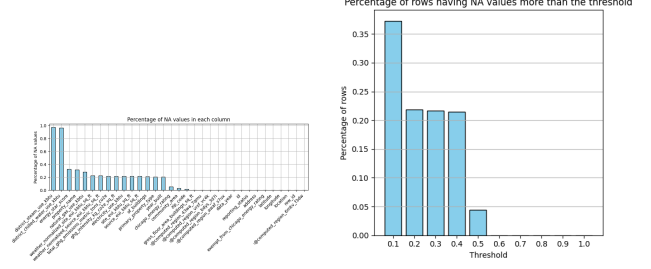


Figure 2. Missing cell percentages by Column (left) and by Row sample threshold (right). We plot the data regarding empty values for each column in descending order. Looking at the chart, the first two bars on the left side of the diagram are almost entirely full of NA values, so we decided to delete them directly. However, because new columns full of NA values might appear in the future, we are considering performing automatic column deletion. If a column contains 40% NA values compared to its length, we will delete it directly. Following the same logic, for each row, if it has more than 40% of NA value compared to its length, we drop it directly.

and availability should be carefully evaluated before deciding whether to include it in the model.

How. Test R^2 drops from 0.977 to ≈ 0.581 . This is the correct number: lower, but it actually generalises.

2.3. v2: Quantile Discretization

What. For each surviving continuous numeric column, bin into $q = 5$ quantiles with `pd.qcut` (Fig. 3).

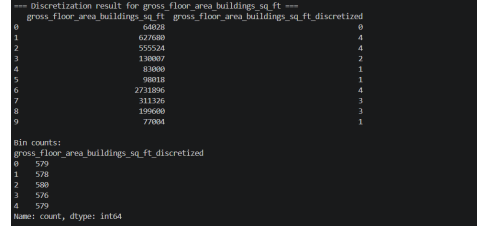


Figure 3. 5-quantile discretization of `gross_floor_area`.

Why. We are using this technique to see if adding extra features can improve performance. Additionally, discretising continuous data allows us to generate further categorical features from those continuous datasets.

How. Test $R^2 \approx 0.543$ (slightly down from v1) but test MAE decreased. v2 is kept because (i) it noticeably reduces test MAE and (ii) it gives v4's feature selection more candidates which might be selected in the feature importance detection.

2.4. v3: Min-Max Normalization

What. Apply Min-Max scaling to every numeric feature:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min} + \epsilon}, \quad \epsilon = 10^{-8}. \quad (2)$$

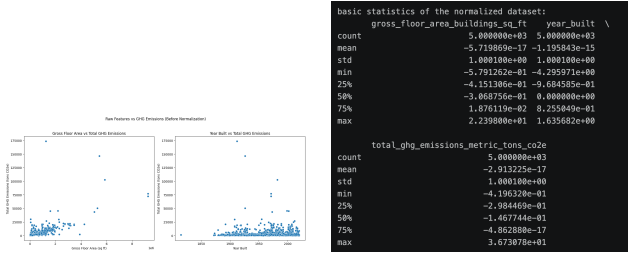


Figure 4. Effect of feature normalization on the two strongest predictors and the GHG label. **Left:** raw scatter plots of `gross_floor_area_buildings_sq_ft` and `year_built` against GHG label. The three axes operate on incompatible scales – floor area spans $0\text{--}9 \times 10^6$, year built spans 1815–2020, and emissions span 0–175,000 tonnes – so a linear model on the raw values would let floor area dominate the loss. **Right:** summary statistics of the same features after normalization, confirming that every feature is rescaled to a common centre and spread (mean ≈ 0 , std ≈ 1); coefficients in the OLS model are now directly comparable.

The training-set min/max are stored in the model artifact and reapplied at inference.

Why. Raw columns differ by 4–5 orders of magnitude (floor area in 10^5 vs. year built in $\sim 2 \times 10^3$, Fig. 4 left). Without scaling: (i) large-scale features dominate the OLS loss, (ii) gradient methods oscillate on elongated contours, (iii) L1/L2 penalties become incomparable across features. Min-Max preserves bounds, which matches physical dimensions better than z-score.

How. For pure OLS the prediction is scale-invariant, so test R^2 moves negligibly (≈ 0.540). The real value is structural: coefficients become comparable, regularised models become possible later, and the v4 importance ranking is no longer biased by column scale, see Fig. 5).

2.5. v4: Importance-Ranked Sequential Forward Selection

What. Rank the v3 features by Gradient Boosting feature importance (Fig. 6), then run Sequential Forward Selection (SFS) on a held-out validation slice of the training set. SFS :1. Start with the top feature, train the model on the training set, and then validate it on the validation set. 2. Add the second feature and perform the same pipeline to get the performance with another feature. 3. Continue this process until the performance on the validation set no longer improves significantly ($\Delta R^2 \leq 0.01$).

Why. v1–v3 leave ~ 20 features feeding OLS, which overfits on a small training set (train $R^2 \approx 0.68$, test $R^2 \approx 0.55$). A threshold-based stop on a separate split is more robust than picking a hardcoded top- K . In the enterprise case, performance is not the only metric. We need to consider other factors regarding economics. For example, we need to consider how to make each inference

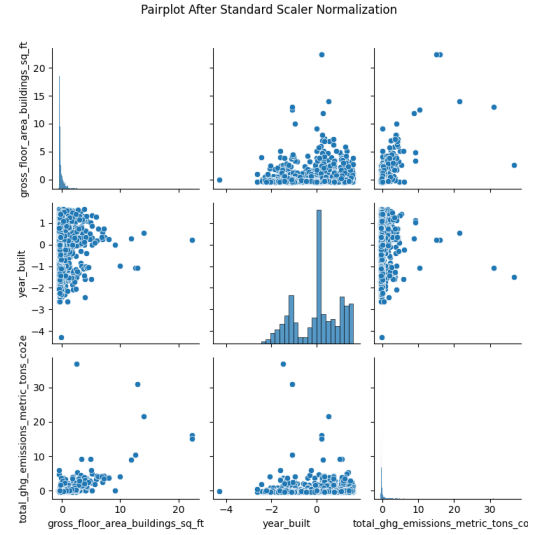


Figure 5. Pairplot of `gross_floor_area_buildings_sq_ft`, `year_built`, and GHG label after Standard Scaler normalization. The diagonal histograms make the underlying right-skew explicit: floor area and emissions both pile up near zero with a long thin tail of large commercial complexes, while year-built is bimodal (a pre-war stock and a post-1970s stock). Off-diagonal scatters show that even after rescaling, the relationships are non-linear and outlier-driven – which is exactly what motivates v2 (quantile discretization, to dampen the tail) and v4 (feature selection, to stop OLS from latching onto spurious correlations among the noisy low-emission majority).

cheaper while maintaining quality. Starting from this point, we adapt feature importance detection in order to reduce the dimension of input to reduce the cost per API call. From another perspective, this can improve the user experience and make the product more competitive because it is cheaper. Stability is also another very important factor. The less information we rely on, the more stable our API is, especially when the quality of input cannot be guaranteed.

How. SFS picks 2 features (Fig. 7):

1. `gross_floor_area_buildings_sq_ft`: val $R^2 = 0.5677$.
2. `weather_normalized_source_eui_kbtu_sq_ft`: val $R^2 = 0.5790$ ($\Delta = +0.0113$). **Accepted.**
3. `primary_property_type`: val $R^2 = 0.5791$ ($\Delta = +0.0001 \leq 0.01$). **Stopped.**

Test R^2 jumps to 0.6882 and test MAE drops to 756.68 – the best non-leaking result.

3. Secondary Improvement: Experimental Tracking

What. We integrated Weights & Biases into `train.py` behind a `--use_wandb` flag and a `--version` name. Each run logs version name and performance metrics.

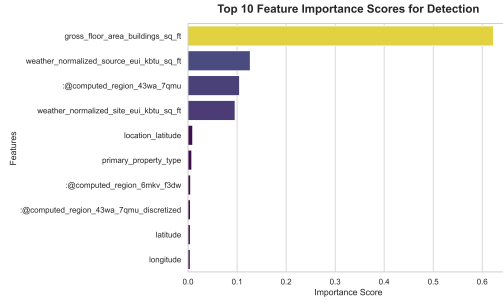


Figure 6. Gradient Boosting feature importance over the v3 feature set. The horizontal axis represents the importance scores, while the vertical axis shows the feature names. From the image, we can see that one specific feature has a very high score. While a few other features show relatively high importance, the rest are quite low. Surprisingly, the location data also has a very low importance score. We will subsequently use the feature importance rankings from this to select the smallest subset of features that maximises model performance.

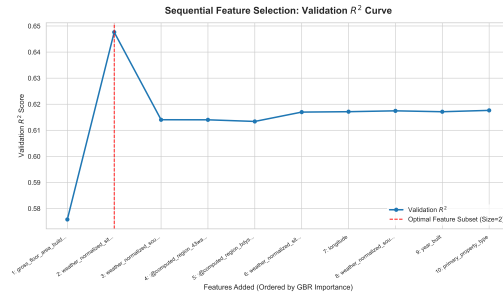


Figure 7. Sequential Forward Selection (SFS) validation R^2 curve. Candidate features are added one at a time in the order given by Gradient Boosting feature importance (x-axis). The curve peaks at **step 2**, after which adding more features causes validation R^2 to drop and then plateau – a clear signal that the extra columns add noise rather than signal. The red dashed line marks the optimal subset size selected by the $\Delta R^2 \leq 0.01$ stopping rule, giving the final 2-feature model used in v4.

Why. We want to monitor the performance tendency over version iterations.

How. The dashboard (Fig. 8) renders the six runs side by side on R^2 , MSE, and MAE for both splits. Two project decisions were made directly from this view: (i) rejecting v0 as leaking despite its top metric, (ii) stopping at v4 because the test MAE bar was clearly the shortest. The dashboard also answers the prompt’s three questions explicitly: *many models?* they live as named runs; *pick the best?* sort by test R^2 / test MAE then sanity-check against baseline to rule out leakage; *metrics?* R^2 for goodness-of-fit, MSE for the loss actually minimised, MAE for an mean absolute error in tonnes of CO₂e.

4. Results

Table 1 summarises the six WandB runs on train and test splits. The v4 pipeline lifts test R^2 by +0.156 over the baseline (a 29% relative improvement) and cuts test MAE by ~ 113 tonnes CO₂e. From v1 to v4, the model gets better on the training set but worse on the test set. This is overfitting: there are too many features. SFS solves it by removing features one by one, until the test score improves.

5. Discussion

What worked. Our project’s feature engineering is highly automated. We don’t manually specify which rows or columns to delete; instead, we only remove them when the null value count exceeds a certain threshold. This approach is designed to account for future uncertainties. Feature selection is also automated. Rather than simply picking the top few features, we start with the single best and most important feature and add others step by step, continuing only until performance stops improving.

What didn’t. Discretization (v2) and normalization (v3) hardly moved test R^2 in isolation; They didn’t deliver the performance gains we expected, but we don’t know why, as we haven’t had the time to investigate the underlying cause.

Limitations. (1) The model is OLS, which assumes additive linear effects; a Gradient Boosting Regressor would likely capture the non-linearities visible in Fig. 4. (2) Mean imputation collapses variance; a model-based imputer for `energy_star_score` is the natural v5, blocked here because that column itself is unavailable at inference time.

If we had more time. (i) Add v5 with better model Gradient Boosting + the same SFS gate; (ii) wire the WandB Model Registry so the API loads “current best” rather than a local `.joblib`; (iii) add a drift-detection job over the yearly Chicago snapshots and re-run the pipeline automatically when columns appear/disappear.

Bonus: Extra Discovery. We discovered during our experiments that data columns tend to fluctuate when fetching data by year. Our findings show that from 2019 to 2022, the data consistently had 28 columns. However, in 2023, this increased to 29 columns(Fig. 9);

This provided us with an insight for real-world applications: the data columns we receive may vary, and the dimensions of those columns might shift as the data updates. To achieve a high level of automation, we need to adapt our missing value processing logic.

Previously, we observed that the two columns with the highest null counts were empty, so we used hard-coding to delete those specific columns while keeping the rest. Moving forward, if new columns are introduced that contain a high volume of null values after a period of data collection, those need to be removed as well.

To automate this process, we should move away from



Figure 8. WandB dashboard: six runs (baseline, v0–v4) compared on every metric. v0 dominates trivially (target leakage); v4 dominates the non-leaking field and is the selected configuration.

Table 1. Metrics for the six WandB runs. v0 leaks (target reconstruction); v4 is the selected non-leaking configuration.

Run	Split	MSE	MAE	R^2	Status
baseline (1 feature)	train	19,915,195.1	1,047.14	0.497	reference
	test	3,852,960.65	869.86	0.532	reference
v0 (all numerics, no filter)	train	654,091.58	226.55	0.984	leaking, rejected
	test	189,767.92	191.150	0.977	
v1 (+ leakage filter, missingness)	train	14,219,021.72	1,312.44	0.641	accepted
	test	3,451,890.74	1,118.48	0.581	
v2 (+ discretization)	train	13,504,978.18	1,143.53	0.659	accepted
	test	3,766,125.95	970.29	0.543	
v3 (+ normalization)	train	13,452,922.16	1,157.61	0.660	accepted
	test	3,790,440.77	990.18	0.540	
v4 (+ SFS, 2 features)	train	17,223,834.55	956.45	0.565	selected
	test	2,568,568.22	756.68	0.688	

hard-coding specific columns for deletion. Instead, we should implement a threshold-based approach:

1. For missing values: Set a threshold where any column is automatically deleted if its null value count exceeds that limit.

2. For categorical data: Apply a similar logic to categorical data deletion. If a column contains categorical data with a number of unique categories that exceeds a certain percentage of the total length, it should be automatically removed rather than relying on hard-coded rules.

This way of handling new features might cause data leakage. When there are new features, engineers should examine the meaning of the features first. If it doesn't cause data leakage, they can leave the rest of the work to the system, and the system will then automatically evaluate the new features and decide whether to use them or not, which is performed by automatic feature importance detection.

In conclusion, instead of using fixed features, the system could handle new features automatically, only provided engineers take a quick check of the meaning of new features.

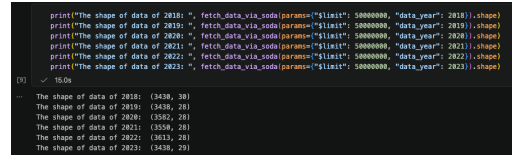


Figure 9. Schema drift: column count per data year. Rows stay around 3.4k–3.6k, but columns swing $30 \rightarrow 28 \rightarrow 29$ over 2018–2023. Hardcoded column lists silently break each time the portal changes, which is why all preprocessing in Section 2 is threshold-based, not name-based.